

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY

Declaration

Plagiarism is defined as “the unacknowledged use, as one’s own work, of work of another person, whether or not such work has been published” (Regulations Governing Conduct at Examinations, 1997, Regulation 1 (viii), University of Malta).

I / We*, the undersigned, declare that the [assignment / Assigned Practical Task report / Final Year Project report] submitted is my / our* work, except where acknowledged and referenced.

I / We* understand that the penalties for making a false declaration may include, but are not limited to, loss of marks; cancellation of examination results; enforced suspension of studies; or expulsion from the degree programme.

Work submitted without this signed declaration will not be corrected, and will be given zero marks.

* Delete as appropriate.

(N.B. If the assignment is meant to be submitted anonymously, please sign this form and submit it to the Departmental Officer separately from the assignment).

David Farrugia

Student Name

Signature



Student Name

Signature

Student Name

Signature

Student Name

Signature

ICT5101

Course Code

ICT5101_IoT_Project_PartA_DavidFarrugia.zip

Title of work submitted

24/05/2026

Date

Remote Plant Monitor System

David Farrugia

Abstract—This report discusses a remote plant monitoring system developed using an arduino nano, a soil moisture sensor, an RGB LED, an OLED display and a Python dashboard. The algorithm reads the soil moisture level at fixed intervals, converts the analogue sensor value into a percentage and alerts the user when the moisture level is low. Local feedback is provided through the OLED display and RGB LED, while the Python dashboard displays the latest reading, plots the moisture values over time, and shows daily and weekly averages, making for a more interactive user interface. Testing highlighted that some instability was present when the moisture sensor was used in actual soil and could still be a limiting factor for this implementation.

Index Terms—IoT, Arduino, python, Automation, Moisture Detection System

I. INTRODUCTION

THIS report outlines the design and implementation of a remote plant monitoring system developed for the ICT5101 Internet of Things assignment. The purpose of the system is to monitor soil moisture and alert the user when the plant requires watering. This was achieved by using an Arduino board connected to a moisture sensor, an RGB LED and an OLED display. The Arduino samples the soil moisture value at fixed intervals, converts the analogue reading into a percentage value, displays the result locally, and sends the reading to the computer through serial communication.

Additionally, along side the arduino-based system, a python dashboard was also developed and used as the computer-side interface. The dashboard receives moisture readings from the Arduino, stores them with timestamps, plots the soil moisture trend over time and displays the daily and weekly averages. Moreover, a threshold is used to determine whether watering is required, with the system flagging dry conditions through the LED and dashboard status.

In this report, the selected hardware, system design, Arduino and python algorithms and testing procedure used to verify the system operation are discussed.

II. HARDWARE SELECTION AND JUSTIFICATION

The hardware was selected to satisfy the main requirements of the remote plant monitoring system: measuring soil moisture, providing a local watering indicator, and transmitting readings to a suitable interface.

A. Arduino Nano

The arduino nano [1] was selected as the main control unit because of its compact size, breadboard compatibility and sufficient I/O pins. Its small design allowed it to be mounted directly on the breadboard, making the prototype more portable and easier to test. Furthermore, the board

provided more than enough processing capability to read analogue data and transmit the results to the python dashboard over serial communication, while also requiring relatively little power to operate compared to an arduino uno board. It is worth noting that power consumption was not a major concern in this project, since the board was always powered through USB.

B. Soil Moisture Sensor

A three pin soil moisture sensor [2] was selected because it provides a direct analogue signal that can be connected to the Arduino without requiring any potentiometer adjustments. This made the sensor easy to integrate and suitable for quick prototyping. The analogue signal was later converted into a moisture percentage in the arduino IDE. However, one limitation of this specific hardware selection is that the lack of direct sensitivity adjustment may make the readings less stable. A possible solution for this is to replace this sensor with a more reliable one, such as the sensor module from Handson Technology [3].

C. RGB LED

An RGB LED [4] was used as the visual alarm. Although the assignment only required the use of a red LED, the RGB LED was chosen since it allowed the use of different colours for different thresholds. In the implemented algorithm, moisture values less than or equal to 25% are shown in red, values up to 50% are shown in yellow and values above 50% are shown in green. This makes the local indicator more informative since the user can quickly identify whether the soil moisture level is low, medium or high.

D. OLED Display

A 0.96-inch 128×64 OLED display [5] was included to provide local feedback directly on the sensing unit. Although the computer dashboard already displays the measured values, the OLED display allowed the current moisture percentage directly from the prototype. The display was included as an additional component to experiment with, since it provided an opportunity to work with an OLED module for the first time.

E. Additional Components

Additional components, including jumper wires, a breadboard, and the USB cable, were used to assemble and power the prototype. These components support rapid prototyping and allow the circuit to be tested and modified easily during development.

III. SYSTEM DESIGN

The system was designed as a wired prototype consisting of three main parts: the sensing unit, the local feedback indicators and a python dashboard. As shown in Fig. 1, the arduino nano acts as the central controller and is connected to the soil moisture sensor, RGB LED and OLED display through the breadboard.

The soil moisture sensor is used as the main input device. Its signal pin is connected to analogue pin A3 on the arduino nano, while its power and ground pins are connected to the 5V and GND rails respectively. The arduino reads the analogue sensor value and converts it into a moisture percentage in software. This percentage is then used to determine the current soil condition.

Local feedback is provided using both an RGB LED and an OLED display. The RGB LED is connected to digital pins D2, D3 and D4 through 220Ω resistors. These pins control the red, green and blue channels of the LED, allowing the system to display different colours depending on the measured moisture level. Red for low moisture level, yellow for an intermediate level and green for high/acceptable levels. The OLED display has two connections to the board, SDA connected to A4 and SCL connected to A5. SDA transfers data between the Arduino and display, while SCL provides the clock signal used to synchronise communication.

Moreover, the arduino nano is connected to the computer through USB, which was used both for power and for serial communication between the python dashboard and arduino board. Moisture readings are transmitted to the dashboard every 5 seconds, where each reading is stored with a timestamp and plotted over time. The dashboard also refreshes every 5 seconds, updating the latest moisture reading, daily average and weekly average. This allows the system to provide immediate local feedback through the OLED display and RGB LED, while also offering a more detailed monitoring interface on the computer.

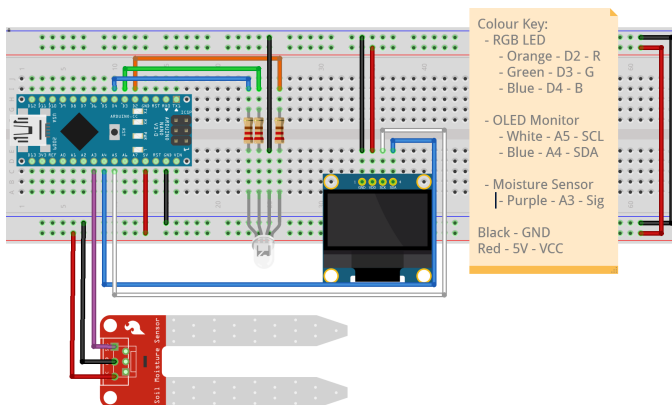


Fig. 1. Circuit diagram of the remote plant monitoring prototype

Lastly, the python dashboard was developed using Plotly Dash [6] and acts as the computer-side monitoring interface for this project. It allows the user to select the serial port and baud rate, connect to the Arduino and view the

incoming moisture readings through an interactive moisture readings over time plot. It also displays summary cards for the latest moisture value, daily and weekly averages and plant status, together with a table of recent logged readings. When the dashboard starts, it checks whether a previous `moisture_history.csv` file exists and loads the saved readings if available. When the user presses the Save & Disconnect button, the recorded data is saved to the same CSV file for later use.

All project files, including the Arduino code, python dashboard and supporting documentation, are available publicly on GitHub¹.

IV. ALGORITHMS DEVELOPED

Two scripts were developed for the project: an arduino sketch which runs locally on the board and a python script that runs on the computer.

The Arduino algorithm controls sensor reading, local feedback, and serial transmission. Every 5 seconds, the analogue value from the soil moisture sensor is read from pin A3 and mapped to a moisture percentage. This percentage is then compared against predefined thresholds to determine the output colour of the RGB LED. The same percentage value is also displayed on the OLED module and transmitted to the computer through serial communication.

On the other hand, the python dashboard algorithm begins once the user selects the correct COM port and presses the connect button. It then attempts to establish serial communication with the Arduino. Once the connection is successful, incoming serial lines are parsed as moisture readings. When a valid reading is received, it is displayed with a timestamp and classified using the same 25% dry threshold. The dashboard then updates the moisture graph, recent readings table, plant status, latest reading, daily average, and weekly average. The daily average is calculated from readings taken on the current date, while the weekly average is calculated from readings recorded during the previous seven days. When the user disconnects, the collected readings are saved to a CSV file.

It is also worth noting that the Arduino IDE Serial Monitor and the python dashboard cannot be used at the same time. Therefore, the Serial Monitor must be closed before running the dashboard. If the port remains unavailable, closing the Arduino IDE or reconnecting the Arduino may solve the issue.

V. TESTING AND RESULTS

Testing was carried out in stages while the system was being developed. Individual components were first connected and tested separately to confirm that each part functioned correctly. The soil moisture sensor, RGB LED, and OLED display were then tested together to verify that the local Arduino-based system responded as expected before moving on to the computer-side dashboard.

The next stage tested the serial connection between the arduino and the python dashboard Fig. 2 shows the dashboard during testing. This was done to confirm that serial communication was a reliable method for transmitting moisture

¹https://github.com/DavidF-22/ICT5101-IoT_Project/tree/main/PartA

readings to the computer. Once the connection was working, the dashboard was developed further and then tested to ensure that the graph, summary cards and recent readings table updated in time with the arduino's sampling interval. During this stage, a glass of water was used to test the moisture sensor and simulate changing moisture values. This provided a clean and simple way to check whether the readings, LED colours, OLED display and dashboard visualisation responded correctly.

One limitation observed here, was that the local Dash web server sometimes terminated execution. When this happened, the dashboard stopped updating and the script had to be executed again. In this case before running the algorithm again, the old browser tab also had to be closed before restarting the dashboard.

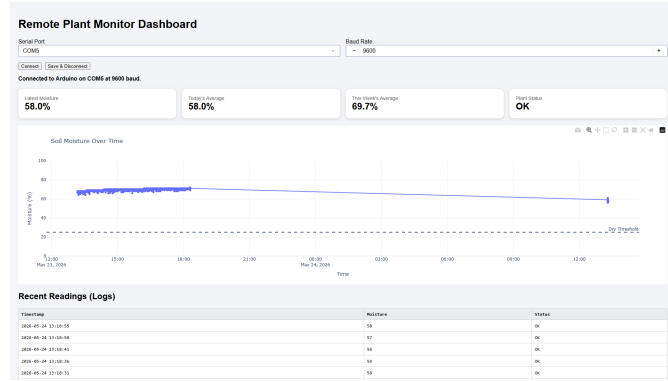


Fig. 2. Python dashboard showing the moisture plot, summary cards and recent readings table during testing.

Later, when a plant was used instead of a glass of water, the instability of the sensor readings became more noticeable. This was expected due to the use of a moisture sensor without a potentiometer to adjust sensitivity. Several approaches were tested to improve stability, including taking multiple readings and calculating their average. However, this still displayed occasional spikes and did not fully solve the issue. The final implementation used a general approach without using thresholds, where the raw analogue reading was scaled to a value between 0% and 100%:

$$MP = \text{constrain} \left(\left(\frac{S}{1023.0} \times 100 \right), 0, 100 \right) \quad (1)$$

where S is the raw analogue sensor value and MP is the calculated moisture percentage. The division by 1023.0 (1023.0 is the maximum analogRead() value on Arduino boards) normalises the Arduino analogue reading, while multiplying by 100 expresses the value as a percentage. The constrain operation ensures that the result remains within the valid 0% to 100% range.

Another limitation observed during testing, was the effect of the soil condition itself. If the soil is hard or compact, inserting the sensor probes can be difficult, and the probes may not make full contact with the surrounding soil. Water may also not spread evenly through the soil, meaning that some areas

near the sensor may remain dry while others become wet. Air gaps or small water pockets around the sensor can also affect the measured value. These factors may cause unstable or misleading readings, even if the sensor and software are working correctly.

VI. CONCLUSION

The remote plant monitoring prototype successfully met the main objectives of the assignment. The chosen arduino was able to read soil moisture values, convert them into a percentage and provide local feedback using the OLED display and RGB LED. The Python dashboard also received the transmitted readings, plotted the moisture values over time, displayed daily and weekly averages, and saved the collected data for later use.

The main limitation observed during testing was the instability of the resistive soil moisture sensor when used in actual soil. This was partly expected due to the lack of direct sensitivity adjustment on the selected sensor. Despite this, the system was able to demonstrate the required monitoring, indication, and visualisation functionality. Future improvements could include using a capacitive moisture sensor for more stable readings, adding wireless communication, and powering the prototype using a battery for a more independent IoT setup.

RESPONSIBLE USE OF GENAI

Generative AI tools were used in this project to help refine the wording, correct grammar and ensure the content remains in scope. The technical design, implementation, testing and conclusions were completed by the author.

REFERENCES

- [1] Mischinti, "Arduino Nano: high-resolution pinout, datasheet and specs," <https://mischianti.org/arduino-nano-high-resolution-pinout-datasheet-and-specs/>.
- [2] IDUINO for maker's life, "Soil Moisture Sensor for Arduino(ME110)," <https://cdn-reichelt.de/documents/datenblatt/A300/ME110.pdf>.
- [3] Handson Technology, "Soil Moisture Sensor Module," <https://handsontec.com/dataspecs/sensor/Soil%20Moisture%20Sensor.pdf>.
- [4] Kingbright, "T-1 3/4 (5mm) FULL COLOR LED LAMP," <https://www.arduino.cc/documents/datasheets/LEDRGB-L-154A4SURK.pdf>.
- [5] TENSTAR ROBOT, "0.96 Inch OLED SSD1306/SSD1315 128x64 IIC I2C Serial Display Module User Manual," <https://manuals.plus/ae/1005005970901119>.
- [6] Plotly, "Dash Documentation," <https://dash.plotly.com/>.